

2. Develop a menu driven Program in C for the following operations on STACK of Integers (Array Implementation of Stack with maximum size MAX)

- a. Push an Element on to Stack
- b. Pop an Element from Stack
- c. Demonstrate how Stack can be used to check Palindrome
- d. Demonstrate Overflow and Underflow situations on Stack
- e. Display the status of Stack
- f. Exit

Support the program with appropriate functions for each of the above operations

Theory:

Stack Operations

1. Stack is a linear data structure that follows the LIFO (Last In, First Out) principle.
2. Push operation inserts an element at the top of the stack, while Pop removes the top element.
3. Overflow occurs when an element is pushed into a full stack, whereas Underflow occurs when an element is popped from an empty stack.
4. Other operations include Peek/Display to view the top element or the current contents of the stack; stacks are also useful for applications such as palindrome checking

Program Solution

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 5

int stack[MAX];
int top = -1;

/* Function declarations */
void push(void);
void pop(void);
void palindrome(void);
void display(void);
void overflow_underflow(void);

/* Push an element onto stack */
void push(void)
{
    int item;

    if (top == MAX - 1)
    {
        printf("\nStack Overflow! Cannot insert more elements.\n");
        return;
    }

    printf("\nEnter the element to push: ");
```

```

scanf("%d", &item);

top++;
stack[top] = item;

printf("%d pushed onto the stack.\n", item);
}

/* Pop an element from stack */
void pop(void)
{
    if (top == -1)
    {
        printf("\nStack Underflow! Stack is empty.\n");
        return;
    }

    printf("\nDeleted element: %d\n", stack[top]);
    top--;
}

/* Display stack elements */
void display(void)
{
    int i;

    if (top == -1)
    {
        printf("\nStack is empty.\n");
        return;
    }

    printf("\nStack elements are:\n");

    for (i = top; i >= 0; i--)
    {
        printf("| %d |\n", stack[i]);
    }

    printf("-----\n");
}

/* Check whether a number is palindrome using stack */
void palindrome(void)
{
    int num, temp, digit;
    int palStack[MAX];
    int palTop = -1;
    int reversed = 0;

    printf("\nEnter a number to check for palindrome: ");

```

```

scanf("%d", &num);

if (num < 0)
{
    printf("Negative numbers are not considered.\n");
    return;
}

temp = num;

/* Push digits onto stack */
while (temp != 0)
{
    digit = temp % 10;

    if (palTop == MAX - 1)
    {
        printf("Number is too large for the demonstration stack.\n");
        return;
    }

    palStack[++palTop] = digit;
    temp = temp / 10;
}

/* Pop digits to construct reverse */
while (palTop != -1)
{
    reversed = reversed * 10 + palStack[palTop--];
}

printf("Original number : %d\n", num);
printf("Reversed number : %d\n", reversed);

if (num == reversed)
    printf("Result: %d is a PALINDROME.\n", num);
else
    printf("Result: %d is NOT a PALINDROME.\n", num);
}

/* Demonstrate Overflow and Underflow */
void overflow_underflow(void)
{
    int choice;

    printf("\n--- Overflow and Underflow Demonstration ---\n");
    printf("1. Demonstrate Overflow\n");
    printf("2. Demonstrate Underflow\n");
    printf("3. Return to Main Menu\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);
}

```

```

switch (choice)
{
    case 1:
        if (top == MAX - 1)
        {
            printf("\nStack is already full.\n");
            printf("Trying to push another element...\n");
            push();
        }
        else
        {
            printf("\nStack is not full yet.\n");
            printf("Push %d elements to demonstrate Overflow.\n", MAX - top - 1);
            printf("After the stack becomes full, select Push again.\n");
        }
        break;

    case 2:
        if (top == -1)
        {
            printf("\nStack is already empty.\n");
            printf("Trying to pop an element...\n");
            pop();
        }
        else
        {
            printf("\nFirst pop all elements from the stack.\n");
            printf("Then select Pop again to demonstrate Underflow.\n");
        }
        break;

    case 3:
        return;

    default:
        printf("\nInvalid choice.\n");
}
}

/* Main function */
int main(void)
{
    int choice;

    while (1)
    {
        printf("\n=====");
        printf("\n    STACK OPERATIONS MENU");
        printf("\n=====");
        printf("\n1. Push an Element");
    }
}

```

```

printf("\n2. Pop an Element");
printf("\n3. Check Palindrome");
printf("\n4. Demonstrate Overflow/Underflow");
printf("\n5. Display Stack");
printf("\n6. Exit");
printf("\n=====");
printf("\nEnter your choice: ");
scanf("%d", &choice);

switch (choice)
{
    case 1:
        push();
        break;

    case 2:
        pop();
        break;

    case 3:
        palindrome();
        break;

    case 4:
        overflow_underflow();
        break;

    case 5:
        display();
        break;

    case 6:
        printf("\nExiting the program...\n");
        exit(0);

    default:
        printf("\nInvalid choice! Please try again.\n");
}
}

return 0;
}

```

Output

Output to be Written in record in blank side.

Inference